

Q1: 给定一个整数数组 nums 和一个整数目标值 target，请你在该数组中找出 和为目标值 target 的那两个 整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案，并且你不能使用两次相同的元素。

你可以按任意顺序返回答案。

解析：哈希表空间换时间

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:
        hashmap = {}

        for i,item in enumerate(nums):
            if target - item in hashmap:
                return i,hashmap[target - item]
            hashmap[item] = i
```

Q2: 给你一个字符串数组，请你将 字母异位词 组合在一起。可以按任意顺序返回结果列表。

解析：字符串的排序以及哈希表的链接

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        hashmap = defaultdict(list)
        for str in strs:
            sort_str = ''.join(sorted(str))
            hashmap[sort_str].append(str)
        return list(hashmap.values())
```

Q3: 给定一个未排序的整数数组 nums , 找出数字连续的最长序列 (不要求序列元素在原数组中连续) 的长度。

请你设计并实现时间复杂度为 $O(n)$ 的算法解决此问题。

解析: 数组去重后利用哈希表的一次访问可以得到最长连续序列, 同时需要排除重复访问 (x-1不在还行表中)

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        longest = 0
        num_set = set(nums)

        for num in num_set:
            if num - 1 not in num_set:
                con = 1
                temp = num
                while temp + 1 in num_set:
                    con = con + 1
                    temp = temp + 1
                longest = max(longest, con)
        return longest
```

Q4:: 给定一个数组 nums，编写一个函数将所有 0 移动到数组的末尾，同时保持非零元素的相对顺序。

解析：移动问题可以看成覆盖问题吗？双指针或者采用覆盖补0

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        j = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[i], nums[j] = nums[j], nums[i]
                j += 1
        return nums
```

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        j = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[j] = nums[i]
                j += 1
        for i in range(j, len(nums)):
            nums[i] = 0
        return nums
```

Q5: 给定一个长度为 n 的整数数组 `height`。有 n 条垂线，第 i 条线的两个端点是 $(i, 0)$ 和 $(i, \text{height}[i])$ 。

找出其中的两条线，使得它们与 x 轴共同构成的容器可以容纳最多的水。

返回容器可以储存的最大水量。

解析：关键在于Area的公式，其要求在距离减小的情况下，必须保证移动的边在变大

```
class Solution:
    def maxArea(self, height: List[int]) -> int:
        maxArea = 0
        i = 0
        j = len(height) - 1
        while i < j:
            Area = min(height[i], height[j]) * (j - i)
            maxArea = max(maxArea, Area)
            if height[i] < height[j]:
                i += 1
            else:
                j -= 1
        return maxArea
```

Q6: 给你一个整数数组 nums，判断是否存在三元组 [nums[i], nums[j], nums[k]] 满足 $i \neq j$ 、 $i \neq k$ 且 $j \neq k$ ，同时还满足 $nums[i] + nums[j] + nums[k] == 0$ 。请你返回所有和为 0 且不重复的三元组。

注意：答案中不可以包含重复的三元组

解释：与两数和类似，当固定一个数a时，使用两数和找sum=-a的数对，必须要求能排序。需要注意的排除重复，利用重复以及边界条件。

注意：nums.sort()是原地排序，返回的就是None。

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        list = []
        nums.sort()
        n = len(nums)
        for i in range(n):
            if i > 0 and nums[i] == nums[i-1]:
                continue
            target = -nums[i]
            r = n - 1
            for l in range(i+1, n):
                if l > i+1 and nums[l] == nums[l-1]:
                    continue
                while l < r and nums[l] + nums[r] > target:
                    r = r - 1
                if l == r:
                    break
                if nums[l] + nums[r] == target:
                    list.append([nums[i], nums[l], nums[r]])

        return list
```

Q7: 给定 n 个非负整数表示每个宽度为 1 的柱子的高度图，计算按此排列的柱子，下雨之后能接多少雨水。

解析：每一个格子的左边和右边必须均高于自己，才有可能在自己的头上接水。

注意：动态规划：动态更新自己左手和右手最高的柱子

```
for j in range(n-2, -1, -1): #从n-2开始遍历到-1 (-1不包括)，且步进为-1
```

```
class Solution:
    def trap(self, height: List[int]) -> int:
        n = len(height)
        water = 0
        max_l = [0] * n
        for i in range(1, n):
            max_l[i] = max(max_l[i-1], height[i-1])
        max_r = [0] * n
        for j in range(n-2, -1, -1):
            max_r[j] = max(max_r[j+1], height[j+1])
        for i in range(n):
            if min(max_l[i], max_r[i]) > height[i]:
                water += min(max_l[i], max_r[i]) - height[i]
        return water
```

Q8: 给定一个字符串 s , 请你找出其中不含有重复字符的 最长子串 的长度。

解析: 字符串子串匹配使用滑动窗口

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        if not s:
            return 0
        s_set = set()
        n = len(s)
        r = 0
        ans = 1
        for l in range(n):
            if l != 0:
                s_set.remove(s[l-1])
            while r < n and s[r] not in s_set:
                s_set.add(s[r])
                r += 1
            ans = max(ans, r - l)
        return ans
```

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        sub = ""
        res = 0
        for ch in s:
            if ch in sub:
                # 找到重复位置, 把重复点之前部分截掉
                idx = sub.index(ch)
                sub = sub[idx + 1:]
            sub += ch
            res = max(res, len(sub))
        return res
```


Q9: 给定两个字符串 s 和 p, 找到 s 中所有 p 的 异位词 的子串, 返回这些子串的起始索引。不考虑答案输出的顺序。

解析: 字符串的匹配, 异位词查询可采用滑动窗口

注意: Counter(s)统计字符串s中所有字母出现的次数, 返回值为字典

```
# len(p) = right - left + 1, => left = right - len(p) + 1 可以用来作为滑动窗口的起  
left = right - len(p) + 1  
if left < 0: # 窗口长度不足 len(p)  
    continue
```

请选择 Python3 提交代码, 而不是 Python

class Solution:

```
def findAnagrams(self, s: str, p: str) -> List[int]:  
    cnt_p = Counter(p) # 统计 p 的每种字母的出现次数  
    cnt_s = Counter() # 统计 s 的长为 len(p) 的子串 t 的每种字母的出现次数  
    ans = []  
  
    for right, c in enumerate(s):  
        cnt_s[c] += 1 # 右端点字母进入窗口  
  
        left = right - len(p) + 1  
        if left < 0: # 窗口长度不足 len(p)  
            continue  
  
        if cnt_s == cnt_p: # t 和 p 的每种字母的出现次数都相同  
            ans.append(left) # t 左端点下标加入答案  
  
        cnt_s[s[left]] -= 1 # 左端点字母离开窗口  
  
    return ans
```

请选择 Python3 提交代码, 而不是 Python

class Solution:

```
def findAnagrams(self, s: str, p: str) -> List[int]:  
    cnt = Counter(p) # 统计 p 的每种字母的出现次数  
    ans = []  
  
    left = 0  
    for right, c in enumerate(s):  
        cnt[c] -= 1 # 右端点字母进入窗口  
        while cnt[c] < 0: # 字母 c 太多了  
            cnt[s[left]] += 1 # 左端点字母离开窗口  
            left += 1  
        if right - left + 1 == len(p): # t 和 p 的每种字母的出现次数都相同 (证明见上)  
            ans.append(left) # t 左端点下标加入答案
```

return ans

Q10: 给你一个整数数组 nums 和一个整数 k，请你统计并返回该数组中和为 k 的子数组的个数。

子数组是数组中元素的连续非空序列。

解析：子数组问题可以采用前缀和求解

注意：只要出现两数和的公式（或者两数相减等于一个值）均可采用哈希表优化

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        n = len(nums)
        count = 0
        sum_nums = 0
        hashmap = {}
        for i in range(n):
            if sum_nums in hashmap:
                hashmap[sum_nums] += 1
            else:
                hashmap[sum_nums] = 1
            sum_nums += nums[i]
            target = sum_nums - k
            if target in hashmap:
                count += hashmap[target]
        return count
```

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:
        cnt = defaultdict(int)
        ans = s = 0
        for x in nums:
            cnt[s] += 1
            s += x
            ans += cnt[s - k]
        return ans
```

Q11: 给你一个整数数组 nums，有一个大小为 k 的滑动窗口从数组的最左侧移动到数组的最右侧。你只可以看到在滑动窗口内的 k 个数字。滑动窗口每次只向右移动一位。

返回滑动窗口中的最大值

解析:

```
import collections

class Solution:
    def maxSlidingWindow(nums, k):
        if not nums or k == 0: return []
        deque = collections.deque()
        # 未形成窗口
        for i in range(k):
            while deque and deque[-1] < nums[i]:
                deque.pop()
            deque.append(nums[i])
        res = [deque[0]]
        # 形成窗口后
        for i in range(k, len(nums)):
            if deque[0] == nums[i - k]:
                deque.popleft()
            while deque and deque[-1] < nums[i]:
                deque.pop()
            deque.append(nums[i])
            res.append(deque[0])
            print(deque)
        return res

print(Solution.maxSlidingWindow([1,3,-1,-3,5,3,6,7], 3))
```

Q12: 给定两个字符串 s 和 t，长度分别是 m 和 n，返回 s 中的 最短窗口 子串，使得该子串包含 t 中的每一个字符（包括重复字符）。如果没有这样的子串，返回空字符串 ""。

测试用例保证答案唯一。

解析：

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        n = len(s)
        l = 0
        ans_l = -1
        ans_r = n-1
        t_count = Counter(t)
        s_count = Counter()
        for r in range(n):
            s_count[s[r]] += 1

            while s_count >= t_count:
                if ans_r - ans_l > r - l:
                    ans_r, ans_l = r, l
                s_count[s[l]] -= 1
                l += 1
        if ans_l < 0:
            return ''
        else:
            return s[ans_l: ans_r + 1]
```

Q13: 给你一个整数数组 `nums` , 请你找出一个具有最大和的连续子数组 (子数组最少包含一个元素) , 返回其最大和。

子数组是数组中的一个连续部分。

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [0] * n
        dp[0] = nums[0]
        for i in range(1,n):
            if dp[i - 1] >= 0:
                dp[i] = dp[i - 1] + nums[i]
            else:
                dp[i] = nums[i]
        return max(dp)
```

Q14: 以数组 `intervals` 表示若干个区间的集合，其中单个区间为 `intervals[i] = [starti, endi]`。请你合并所有重叠的区间，并返回 一个不重叠的区间数组，该数组需恰好覆盖输入中的所有区间。

解析：当前区间左端在前一个区间的范围内时，执行插入，需要外层的List按左端排序。

注意：`intervals.sort(key = lambda x:x[0])` 可以有效降低排序时间

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort(key = lambda x:x[0])
        ans = []
        for interval in intervals:
            if not ans or ans[-1][1] < interval[0]:
                ans.append(interval)
            else:
                ans[-1][1] = max(ans[-1][1], interval[1])
        return ans
```

Q15: 给定一个整数数组 nums，将数组中的元素向右轮转 k 个位置，其中 k 是非负数。

解析: $a^{-1} b^{-1})^{-1} = ba$

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        q = deque(nums)
        for _ in range(k % len(nums)):
            q.appendleft(q.pop())
        nums[:] = list(q)
```

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        nums.reverse()
        k = k % len(nums)
        nums[:k] = reversed(nums[:k])
        nums[k:] = reversed(nums[k:])
```


Q16: 给你一个整数数组 nums，返回 数组 answer，其中 answer[i] 等于 nums 中除了 nums[i] 之外其余各元素的乘积。

题目数据 保证 数组 nums 之中任意元素的全部前缀元素和后缀的乘积都在 32 位 整数范围内。

请 不要使用除法，且在 O(n) 时间复杂度内完成此题。

解析：前缀和变成前缀积

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        n = len(nums)
        pre_list = [1] * n
        suf_list = [1] * n
        ans = []
        for i in range(1, n):
            pre_list[i] = pre_list[i - 1] * nums[i - 1]
        for i in range(n - 2, -1, -1):
            suf_list[i] = suf_list[i + 1] * nums[i + 1]
        for i in range(n):
            ans.append(pre_list[i] * suf_list[i])
        return ans
```

Q17: 给你一个未排序的整数数组 `nums` , 请你找出其中没有出现的最小的正整数。(不严谨)

请你实现时间复杂度为 $O(n)$ 并且只使用常数级别额外空间的解决方案。

解析: 最坏情况是`nums`包含 1 到 `len(nums)` 的所有数, 所以开辟`set`和哈希表的最坏情况是 $O(n)$ 。

```
class Solution:
    def firstMissingPositive(self, nums: List[int]) -> int:
        nums = set(nums)
        n = len(nums)
        for i in range(1, n + 1):
            if i in nums:
                continue
            else:
                return i
        return i + 1
```

Q18: 给定一个 $m \times n$ 的矩阵，如果一个元素为 0，则将其所在行和列的所有元素都设为 0。请使用原地算法。

解析：记录0出现的行和列号。然后使用哈希表优化空间

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        m = len(matrix)
        n = len(matrix[0])
        m_list = defaultdict(int)
        n_list = defaultdict(int)
        for i in range(m):
            for j in range(n):
                if matrix[i][j] == 0:
                    m_list[i] = 1
                    n_list[j] = 1
        for i in range(m):
            for j in range(n):
                if m_list[i] == 1:
                    matrix[i][j] = 0
                if n_list[j] == 1:
                    matrix[i][j] = 0
```

Q19: 给你一个 m 行 n 列的矩阵 matrix，请按照 顺时针螺旋顺序，返回矩阵中的所有元素。

解析：四指针一起跑

```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        m = len(matrix)
        n = len(matrix[0])
        up = left = 0
        down = m - 1
        right = n - 1

        ans = []

        while len(ans) < m * n:
            print(ans)
            for i in range(left, right + 1):
                if len(ans) == m * n:
                    return ans
                ans.append(matrix[up][i])
            up += 1

            for i in range(up, down + 1):
                if len(ans) == m * n:
                    return ans
                ans.append(matrix[i][right])
            right -= 1

            for i in range(right, left - 1, -1):
                if len(ans) == m * n:
                    return ans
                ans.append(matrix[down][i])
            down -= 1

            for i in range(down, up - 1, -1):
                if len(ans) == m * n:
                    return ans
                ans.append(matrix[i][left])
            left += 1
        return ans
```

Q20: 给定一个 $n \times n$ 的二维矩阵 matrix 表示一个图像。请你将图像顺时针旋转 90 度。

你必须在 原地 旋转图像，这意味着你需要直接修改输入的二维矩阵。请不要 使用另一个矩阵来旋转图像。

解析：反转加逆

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        # 水平翻转
        for i in range(n // 2):
            for j in range(n):
                matrix[i][j], matrix[n - i - 1][j] = matrix[n - i - 1][j], matrix[i][j]
        # 主对角线翻转
        for i in range(n):
            for j in range(i):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]
```

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        n = len(matrix)
        for i in range(n // 2):
            for j in range((n + 1) // 2):
                (
                    matrix[i][j],
                    matrix[n - j - 1][i],
                    matrix[n - i - 1][n - j - 1],
                    matrix[j][n - i - 1],
                ) = (
                    matrix[n - j - 1][i],
                    matrix[n - i - 1][n - j - 1],
                    matrix[j][n - i - 1],
                    matrix[i][j],
                )
```

Q21: 编写一个高效的算法来搜索 $m \times n$ 矩阵 matrix 中的一个目标值 target 。该矩阵具有以下特性:

- 每行的元素从左到右升序排列。
- 每列的元素从上到下升序排列。

解析:

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        m=len(matrix)
        n=len(matrix[0])
        # 从右上角开始寻找
        i,j=0,n-1
        while i<m and j>=0:
            if matrix[i][j]<target:
                i+=1
            elif matrix[i][j]>target:
                j-=1
            else:
                return True
        return False
```

Q22: 给你两个单链表的头节点 headA 和 headB，请你找出并返回两个单链表相交的起始节点。如果两个链表不存在相交节点，返回 null。

图示两个链表在节点 c1 开始相交

解析：尾部对齐。返回地址，p == q

```
class Solution:
    def getIntersectionNode(self, headA: ListNode, headB: ListNode) -> Optional[ListNode]:

        def getLen(head):
            length = 0
            while head:
                length += 1
                head = head.next
            return length

        lenA = getLen(headA)
        lenB = getLen(headB)

        p, q = headA, headB

        # 对齐
        if lenA > lenB:
            for _ in range(lenA - lenB):
                p = p.next
        else:
            for _ in range(lenB - lenA):
                q = q.next

        # 同步走
        while p:
            if p == q:
                return p
            p = p.next
            q = q.next

        return None
```

Q23: 给你单链表的头节点 head , 请你反转链表, 并返回反转后的链表。

注意: 翻转链表要判断链表是不是空

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        p = None
        r = head

        while r:
            r = head.next
            head.next = p
            p = head
            head = r

        return p
```


Q24: 给你一个单链表的头节点 head , 请你判断该链表是否为回文链表。如果是, 返回 true ; 否则, 返回 false 。

解析: 判断中点, 从中间比较两边

```
class Solution:
    def isPalindrome(self, head: Optional[ListNode]) -> bool:
        p = head
        r = head
        nums = []
        while r and r.next:
            nums.append(p.val)
            p = p.next
            r = r.next.next
        if r != None:
            p = p.next
        for i in range(len(nums)-1,-1,-1):
            if nums[i] != p.val:
                return False
            else:
                p = p.next
        return True
```

Q25: 给定一个链表的头节点 head , 返回链表开始入环的第一个节点。 如果链表无环, 则返回 null。

解析: 快慢指针

```
class Solution(object):
    def detectCycle(self, head):
        fast, slow = head, head
        while True:
            if not (fast and fast.next): return
            fast, slow = fast.next.next, slow.next
            if fast == slow: break
        fast = head
        while fast != slow:
            fast, slow = fast.next, slow.next
        return fast
```

Q26: 将两个升序链表合并为一个新的 升序 链表并返回。新链表是通过拼接给定的两个链表的所有节点组成的。

解析：谁小谁插入

```
def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode]) ->
Optional[ListNode]:
    head = ListNode(-1)
    p = head
    while list1 and list2:
        if list1.val <= list2.val:
            p.next = list1
            list1 = list1.next
        else:
            p.next = list2
            list2 = list2.next
        p = p.next
    if list1:
        p.next = list1
    else:
        p.next = list2

    return head.next
```

Q27: 给你两个 非空 的链表，表示两个非负的整数。它们每位数字都是按照 逆序 的方式存储的，并且每个节点只能存储 一位 数字。

请你将两个数相加，并以相同形式返回一个表示和的链表。

你可以假设除了数字 0 之外，这两个数都不会以 0 开头。

解析：短的部分可以补0

```
def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
    j = i = 0
    num1 = 0
    num2 = 0
    head = ListNode(-1)
    p = head

    while l1:
        num1 = num1 + l1.val * (10 ** i)
        i = i + 1
        l1 = l1.next
    while l2:
        num2 = num2 + l2.val * (10 ** j)
        j = j + 1
        l2 = l2.next
    num = num1 + num2

    while num // 10 != 0:
        p.val = num % 10
        p.next = ListNode(-1)
        p = p.next
        num = num // 10

    p.val = num % 10
    return head
```

```
def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) ->
Optional[ListNode]:
    head = ListNode(-1) # 哑节点
    p = head
    c = 0 # 进位

    # 两个链表同时遍历
    while l1 or l2 or c:
        val1 = l1.val if l1 else 0
        val2 = l2.val if l2 else 0

        total = val1 + val2 + c
        c = total // 10
```

```
p.next = ListNode(total % 10)
p = p.next

if l1: l1 = l1.next
if l2: l2 = l2.next

return head.next
```

Q28：给你一个链表，删除链表的倒数第 n 个结点，并且返回链表的头结点。

解析：快慢指针

```
def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
    fast = slow = head

    # fast 先走 n 步
    while n:
        fast = fast.next
        n -= 1

    # 如果 fast 为 None, 说明删除的是头节点
    if not fast:
        return head.next

    # fast 和 slow 同时走, 直到 fast 到达最后一个节点
    while fast.next:
        fast = fast.next
        slow = slow.next

    # slow.next 就是要删除的节点
    slow.next = slow.next.next

    return head
```

Q29: 给你一个链表，两两交换其中相邻的节点，并返回交换后链表的头节点。你必须在不修改节点内部的值的情况下完成本题（即，只能进行节点交换）。

解析：增加一个头节点保存位置，然后p和r换位置

```
def swapPairs(self, head: Optional[ListNode]) -> Optional[ListNode]:
    dummyHead = ListNode(0)
    dummyHead.next = head
    temp = dummyHead
    if not head:
        return head
    while temp.next and temp.next.next:
        p = temp.next
        r = temp.next.next
        temp.next = r
        p.next = r.next
        r.next = p
        temp = temp.next.next

    return dummyHead.next
```

Q30：给你链表的头节点 head ，每 k 个节点一组进行翻转，请你返回修改后的链表。

k 是一个正整数，它的值小于或等于链表的长度。如果节点总数不是 k 的整数倍，那么请将最后剩余的节点保持原有顺序。

你不能只是单纯的改变节点内部的值，而是需要实际进行节点交换。

解析：不好理解p0和dummy

```
class Solution:
    def reverseKGroup(self, head: Optional[ListNode], k: int) -> Optional[ListNode]:
        n = 0
        pre = head
        while pre:
            n += 1
            pre = pre.next

        p0 = dummy = ListNode(next = head)
        pre = None
        cur = head

        while n >= k:
            n -= k
            for _ in range(k):
                nxt = cur.next
                cur.next = pre
                pre = cur
                cur = nxt
            nxt = p0.next
            nxt.next = cur
            p0.next = pre
            p0 = nxt
        return dummy.next
```


Q31: 给你一个长度为 n 的链表，每个节点包含一个额外增加的随机指针 `random`，该指针可以指向链表中的任何节点或空节点。

构造这个链表的 深拷贝。深拷贝应该正好由 n 个全新节点组成，其中每个新节点的值都设为其对应的原节点的值。新节点的 `next` 指针和 `random` 指针也都应指向复制链表中的新节点，并使原链表和复制链表中的这些指针能够表示相同的链表状态。复制链表中的指针都不应指向原链表中的节点。

例如，如果原链表中有 X 和 Y 两个节点，其中 $X.random \rightarrow Y$ 。那么在复制链表中对应的两个节点 x 和 y ，同样有 $x.random \rightarrow y$ 。

返回复制链表的头节点。

用一个由 n 个节点组成的链表来表示输入/输出中的链表。每个节点用一个 `[val, random_index]` 表示：

- `val`：一个表示 `Node.val` 的整数。
- `random_index`：随机指针指向的节点索引（范围从 `0` 到 `n-1`）；如果不指向任何节点，则为 `null`。

你的代码只接受原链表的头节点 `head` 作为传入参数。

解析：怎么处理 `random`

```
def copyRandomList(self, head: 'Optional[Node]') -> 'Optional[Node]':
    if not head:
        return head
    cur = head

    while cur:
        node = Node(cur.val, next = cur.next)
        cur.next = node
        cur = node.next

    cur = head

    while cur:
        if cur.random:
            cur.next.random = cur.random.next
            cur = cur.next.next

    cur = head
    pre = res = cur.next
    while pre.next:
        cur.next = cur.next.next
        pre.next = pre.next.next
        cur = cur.next
        pre = pre.next
    cur.next = None

    return res
```

Q32: 给你链表的头结点 head , 请将其按 升序 排列并返回 排序后的链表 。

进阶: 你可以在 $O(n \log n)$ 时间复杂度和常数级空间复杂度下, 对链表进行排序吗?

解析: 分治+归并排序或者自底向上直接排序

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        if not head or not head.next:
            return head

        slow, fast = head, head.next
        while fast and fast.next:
            fast, slow = fast.next.next, slow.next

        mid, slow.next = slow.next, None
        left, right = self.sortList(head), self.sortList(mid)
        h = self.mergeTwoLists(left, right)
        return h

    def mergeTwoLists(self, list1, list2):
        head = ListNode(-1)
        p = head
        while list1 and list2:
            if list1.val <= list2.val:
                p.next = list1
                list1 = list1.next
            else:
                p.next = list2
                list2 = list2.next
            p = p.next
        if list1:
            p.next = list1
        else:
            p.next = list2

        return head.next
```

```
def sortList(self, head: Optional[ListNode]) -> Optional[ListNode]:
    n = 0
    cur = head
    while cur:
        n += 1
        cur = cur.next
```

```

dummy = ListNode(-1, head)
step = 1

while step < n:
    cur = dummy.next
    tail = dummy

    while cur:
        left = cur
        right = self.cut(left, step)
        cur = self.cut(right, step)

        merged_head, merged_tail = self.merge(left, right)
        tail.next = merged_head
        tail = merged_tail

    step *= 2

return dummy.next

def cut(self, node, k):
    while node and k - 1 > 0:
        node = node.next
        k -= 1
    if not node:
        return None
    rest = node.next
    node.next = None
    return rest

def merge(self, l1, l2):
    if not l1:
        tail = l2
        while tail and tail.next:
            tail = tail.next
        return l2, tail
    if not l2:
        tail = l1
        while tail and tail.next:
            tail = tail.next
        return l1, tail

    dummy = ListNode(-1)
    p = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            p.next = l1
            l1 = l1.next
        else:
            p.next = l2
            l2 = l2.next
        p = p.next
    p.next = l1 if l1 else l2

```

```
while p.next:  
    p = p.next  
return dummy.next, p
```

Q33: 给你一个链表数组，每个链表都已经按升序排列。

请你将所有链表合并到一个升序链表中，返回合并后的链表

解析：使用堆来比较k个链表的最小值，注意怎么使用堆来排序。

```
ListNode.__lt__ = lambda a, b: a.val < b.val # 让堆可以比较节点大小
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        h = []
        for i in range(len(lists)):
            h.append(lists[i])
        p = dummy = ListNode()
        heapify(h)
        while h:
            node = heappop(h)
            if node.next:
                heappush(h, node.next)
            p.next = node
            p = p.next
        return dummy.next
```

```
class Solution:
    def mergeKLists(self, lists: List[Optional[ListNode]]) -> Optional[ListNode]:
        h = []
        # 把每个链表的头节点加入堆
        for i in range(len(lists)):
            if lists[i]:
                h.append((lists[i].val, i, lists[i])) # ✅ 元组避免节点比较 必须要添加i，不然当
val相同时，堆会比较node，而node是无法比较的。

        p = dummy = ListNode()
        heapify(h)

        while h:
            val, i, node = heappop(h)
            if node.next:
                heappush(h, (node.next.val, i, node.next)) # ✅ 同样存元组
            p.next = node
            p = p.next

        return dummy.next
```

Q34: 请你设计并实现一个满足 LRU (最近最少使用) 缓存 约束的数据结构。

实现 LRU Cache 类:

- `LRUCache(int capacity)` 以 正整数 作为容量 `capacity` 初始化 LRU 缓存
- `int get(int key)` 如果关键字 `key` 存在于缓存中, 则返回关键字的值, 否则返回 `-1`。
- `void put(int key, int value)` 如果关键字 `key` 已经存在, 则变更其数据值 `value`; 如果不存在, 则向缓存中插入该组 `key-value`。如果插入操作导致关键字数量超过 `capacity`, 则应该 **逐出** 最久未使用的关键字。

函数 `get` 和 `put` 必须以 $O(1)$ 的平均时间复杂度运行。

解析: 1.链表+hashmap组合。2.python内置有序字典

```
class ListNode:
    def __init__(self, key = None, val = None, pre = None, next = None):
        self.key = key
        self.val = val
        self.pre = pre
        self.next = next

class LRUCache:

    def __init__(self, capacity: int):
        self.capacity = capacity
        self.hashmap = {}

        self.head = ListNode()
        self.tail = ListNode(pre = self.head)
        self.head.next = self.tail

    def get(self, key: int) -> int:
        if key in self.hashmap:
            value = self.hashmap[key].val
            self.move_node_to_tail(key)
            return value
        else:
            return -1

    def put(self, key: int, value: int) -> None:
        if key in self.hashmap:
            self.hashmap[key].val = value
            self.move_node_to_tail(key)
        else:
            if len(self.hashmap) == self.capacity:
                self.hashmap.pop(self.head.next.key)
                self.head.next = self.head.next.next
                self.head.next.pre = self.head
            else:
                new_node = ListNode(key, value)
                self.head.next = new_node
                new_node.pre = self.head
                self.hashmap[key] = new_node
```

```

        node = ListNode(key,value)
        node.pre = self.tail.pre
        node.next = self.tail
        self.tail.pre.next = node
        self.tail.pre = node
        self.hashmap[key] = node

def move_node_to_tail(self, key):
    node = self.hashmap[key]
    node.pre.next = node.next
    node.next.pre = node.pre

    node.pre = self.tail.pre
    node.next = self.tail
    self.tail.pre.next = node
    self.tail.pre = node

```

```
class LRUCache:
```

```

    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()

    def get(self, key):
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)    # 移到尾部
        return self.cache[key]

    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)    # 已存在先移到尾部
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)    # 删除头部 (最旧)

```

Q35: 给定一个二叉树的根节点 root , 返回 它的 中序 遍历 。

解析: 使用递归和迭代都会用栈, 会有额外开销。

```
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    if not root:
        return []
    return self.inorderTraversal(root.left) + [root.val] +
self.inorderTraversal(root.right)
```

```
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        ans = []
        s = deque()
        while root or s:
            if root:
                s.append(root)
                root = root.left
            else:
                node = s.pop()
                ans.append(node.val)
                root = node.right
        return ans
```


Q36: 给一个二叉树 root , 返回其最大深度。

二叉树的 最大深度 是指从根节点到最远叶子节点的最长路径上的节点数。

解析: 高度等于左子树和右子树中最深的值+1

```
def maxDepth(self, root: Optional[TreeNode]) -> int:
    if root is None:
        return 0
    return max(self.maxDepth(root.left), self.maxDepth(root.right)) + 1
```

Q37: 给你一棵二叉树的根节点 root , 翻转这棵二叉树, 并返回其根节点。

解析: 左子树和右子树翻转

```
def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
    if not root:
        return
    node = root.left
    root.left = root.right
    root.right = node
    self.invertTree(root.left)
    self.invertTree(root.right)
    return root
```

Q38: 给你一个二叉树的根节点 root , 检查它是否轴对称。

解析:

```
def issymmetric(self, root: Optional[TreeNode]) -> bool:
    def check(left, right):
        if not left and not right:
            return True
        if not left or not right:
            return False
        if left.val != right.val:
            return False
        return check(left.left, right.right) and check(left.right, right.left)

    return check(root.left, root.right)
```

```
def issymmetric(self, root: Optional[TreeNode]) -> bool:
    q = deque()
    q.append((root.left, root.right))

    while q:
        left, right = q.popleft()

        if not left and not right:
            continue
        if not left or not right:
            return False
        if left.val != right.val:
            return False

        q.append((left.left, right.right))    # 外侧一对
        q.append((left.right, right.left))    # 内侧一对

    return True
```

Q39: 给你一棵二叉树的根节点，返回该树的 直径。

二叉树的 直径 是指树中任意两个节点之间最长路径的 长度。这条路径可能经过也可能不经过根节点 root。
两节点之间路径的 长度 由它们之间边数表示。

解析：经过当前节点的最长直径为左子树的高度加右子树的高度

```
def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
    ans = 0
    def dfs(node):
        nonlocal ans
        if not node:
            return 0
        left = dfs(node.left)    # 左子树高度
        right = dfs(node.right)  # 右子树高度
        ans = max(ans, left + right) # 经过当前节点的直径
        return max(left, right) + 1 # 返回当前节点高度给父节点用
    dfs(root)
    return ans
```

Q40: 给你二叉树的根节点 root , 返回其节点值的 层序遍历 。（即逐层地，从左到右访问所有节点）。

解析：BFS可以层序遍历，但无法输出二位数组，使用flag标记，或者一次处理所有数据节点。

```
def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
    q = deque()
    if not root:
        return []
    q.append(root)
    flag = len(q)
    ans = []
    temp = []
    while flag:
        node = q.popleft()
        temp.append(node.val)
        if node.left:
            q.append(node.left)
        if node.right:
            q.append(node.right)
        flag -= 1
        if flag == 0:
            ans.append(temp)
            temp = []
            flag = len(q)
    return ans
```

Q41: 给你一个整数数组 `nums` , 其中元素已经按 升序 排列, 请你将其转换为一棵 平衡 二叉搜索树。

解析: 这个题出的不好

```
def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:
    if not nums:
        return

    mid = len(nums) // 2
    root = TreeNode(nums[mid])
    root.left = self.sortedArrayToBST(nums[:mid])
    root.right = self.sortedArrayToBST(nums[mid+1:])

    return root
```

Q42: 给你一个二叉树的根节点 root , 判断其是否是一个有效的二叉搜索树。

有效 二叉搜索树定义如下:

- 节点的左子树只包含 严格小于 当前节点的数。
- 节点的右子树只包含 严格大于 当前节点的数。
- 所有左子树和右子树自身必须也是二叉搜索树。

解析: 每个节点要有范围

```
def isValidBST(self, root: Optional[TreeNode]) -> bool:
    def DFS(root):
        if not root:
            return []
        return DFS(root.left) + [root.val] + DFS(root.right)

    nums = DFS(root)
    for i in range(len(nums) - 1):
        if nums[i] >= nums[i+1]:
            return False
    return True
```

```
def isValidBST(self, root: Optional[TreeNode]) -> bool:
    def check(node, min_val, max_val):
        if not node:
            return True
        if node.val <= min_val or node.val >= max_val:
            return False
        return check(node.left, min_val, node.val) and check(node.right, node.val,
max_val)

    return check(root, float('-inf'), float('inf'))
```

Q43: 给定一个二叉搜索树的根节点 root , 和一个整数 k , 请你设计一个算法查找其中第 k 小的元素 (k 从 1 开始计数) 。

```
def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
    def DFS(root):
        if not root:
            return []
        return DFS(root.left) + [root.val] + DFS(root.right)

    nums = DFS(root)
    return nums[k-1]
```

```
def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
    q = deque()
    while root or q:
        if root:
            q.append(root)
            root = root.left
        else:
            node = q.pop()
            k -= 1
            root = node.right
    if k == 0:
        return node.val
```


Q44: 给定一个二叉树的根节点 root，想象自己站在它的右侧，按照从顶部到底部的顺序，返回从右侧所能看到的节点值。

解析：记录层序遍历的每一层的最后一个节点的val

```
def rightSideview(self, root: Optional[TreeNode]) -> List[int]:
    ans = []
    if not root:
        return ans
    q = deque()
    q.append(root)
    while q:
        n = len(q)
        while n:
            node = q.popleft()
            n -= 1
            if node.left:
                q.append(node.left)
            if node.right:
                q.append(node.right)
            if n == 0:
                ans.append(node.val)
    return ans
```

Q45: 给你二叉树的根结点 root , 请你将它展开为一个单链表:

- 展开后的单链表应该同样使用 `TreeNode` , 其中 `right` 子指针指向链表中下一个结点, 而左子指针始终为 `null` 。
- 展开后的单链表应该与二叉树 先序遍历 顺序相同。

解析: 右子树会在左子树的最右边节点之后访问

```
def flatten(self, root: TreeNode) -> None:
    curr = root
    while curr:
        if curr.left:
            predecessor = curr.left
            while predecessor.right:
                predecessor = predecessor.right
            predecessor.right = curr.right
            curr.right = curr.left
            curr.left = None
        curr = curr.right
```

Q46: 给定两个整数数组 preorder 和 inorder，其中 preorder 是二叉树的先序遍历，inorder 是同一棵树的中序遍历，请构造二叉树并返回其根节点

解析

```
def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
    index = {val: i for i, val in enumerate(inorder)} # O(n) 预处理

    def dfs(pre_left, pre_right, in_left, in_right):
        if pre_left > pre_right:
            return None
        root_val = preorder[pre_left]
        root = TreeNode(root_val)
        i = index[root_val] # O(1) 查找
        left_size = i - in_left
        root.left = dfs(pre_left+1, pre_left+left_size, in_left, i-1)
        root.right = dfs(pre_left+left_size+1, pre_right, i+1, in_right)
        return root
    n = len(preorder)
    return dfs(0, n-1, 0, n-1)
```

Q47: 给定一个二叉树的根节点 `root` , 和一个整数 `targetSum` , 求该二叉树里节点值之和等于 `targetSum` 的路径 的数目。

路径 不需要从根节点开始, 也不需要叶子节点结束, 但是路径方向必须是向下的 (只能从父节点到子节点) 。

解析:

```
def pathSum(self, root: Optional[TreeNode], targetSum: int) -> int:
    # key: 从根到 node 的节点值之和
    # value: 节点值之和的出现次数
    # 注意在递归过程中, 哈希表只保存根到 node 的路径的前缀的节点值之和
    cnt = defaultdict(int)
    cnt[0] = 1
    ans = 0

    # s 表示从根到 node 的父节点的节点值之和 (node 的节点值尚未计入)
    def dfs(node: Optional[TreeNode], s: int) -> None:
        if node is None:
            return

        nonlocal ans
        s += node.val
        # 把 node 当作路径的终点, 统计有多少个起点
        ans += cnt[s - targetSum]

        cnt[s] += 1
        dfs(node.left, s)
        dfs(node.right, s)
        cnt[s] -= 1 # 恢复现场 (撤销 cnt[s] += 1)

    dfs(root, 0)
    return ans
```

Q48：给定一个二叉树, 找到该树中两个指定节点的最近公共祖先。

最近公共祖先的定义为：“对于有根树 T 的两个节点 p、q，最近公共祖先表示为一个节点 x，满足 x 是 p、q 的祖先且 x 的深度尽可能大（一个节点也可以是它自己的祖先）。

解析：1.当p, q分别在左子树和右子树能找到时，返回当前root 2.只找到其中一个，则另一个必然包含在它的子树里，即返回找到的p或者q。3.都没找到，返回None

```
def lowestCommonAncestor(self, root: TreeNode, p: TreeNode, q: TreeNode) -> TreeNode:
    if not root or root == p or root == q: return root
    left = self.lowestCommonAncestor(root.left, p, q)
    right = self.lowestCommonAncestor(root.right, p, q)
    if not left: return right
    if not right: return left
    return root
```

Q49: 二叉树中的 路径 被定义为一条节点序列，序列中每对相邻节点之间都存在一条边。同一个节点在一条路径序列中 至多出现一次。该路径 至少包含一个 节点，且不一定经过根节点。

路径和*是路径中各节点值的总和。

给你一个二叉树的根节点 root，返回其 最大路径和。

解析：对一个节点，需要处理两个数据：1.左子树的最大和+当前节点的值+右子树的最大和。2.返回给父节点一个值，这个值时当前节点加上左子树和右子树两者中最大值，且如果小于0则返回0

```
def maxPathSum(self, root: Optional[TreeNode]) -> int:
    ans = -inf
    def dfs(root):
        if not root:
            return 0
        l_val = dfs(root.left)
        r_val = dfs(root.right)
        nonlocal ans
        ans = max(ans, l_val + r_val + root.val)
        return max(max(l_val, r_val) + root.val, 0)
    dfs(root)
    return ans
```

Q50: 给你一个由 '1'（陆地）和 '0'（水）组成的二维网格，请你计算网格中岛屿的数量。

岛屿总是被水包围，并且每座岛屿只能由水平方向和/或竖直方向上相邻的陆地连接形成。

此外，你可以假设该网格的四条边均被水包围。

解析：当前值为1的时候，开始沉没附近的陆地。

```
def numIslands(self, grid: List[List[str]]) -> int:
    nr, nc = len(grid), len(grid[0])
    ans = 0
    q = deque()

    for r in range(nr):
        for c in range(nc):
            if grid[r][c] == '1':
                q.append((r,c))
                grid[r][c] = '0'
                ans += 1
                while q:
                    x, y = q.popleft()
                    for row, col in [(x-1, y), (x,y-1), (x+1, y), (x,y+1)]:
                        if 0 <= row < nr and 0 <= col < nc and grid[row][col] == '1':
                            q.append((row,col))
                            grid[row][col] = '0'

    return ans
```

```
def numIslands(self, grid: List[List[str]]) -> int:
    m = len(grid)
    n = len(grid[0])
    count = 0

    def sink(x, y):
        grid[x][y] = '0'
        if x > 0 and grid[x-1][y] == '1':
            sink(x-1, y)
        if x < m-1 and grid[x+1][y] == '1':
            sink(x+1, y)
        if y < n-1 and grid[x][y+1] == '1':
            sink(x, y+1)
        if y > 0 and grid[x][y-1] == '1':
            sink(x, y-1)

    for x in range(m):
        for y in range(n):
            if grid[x][y] == '1':
                count += 1
                sink(x, y)

    return count
```